

IPSupport **LLC**

Any Model. Your Hardware. Your Data.

How we compress and fine-tune open models — language, vision, speech, image and music — to run on less hardware, faster, and on the client's own tasks. 23 open releases, every number reproducible.

3.9×

less memory: Nemotron 30B
at +2.5% perplexity

2.1×

faster full-duplex voice,
20/20 answers right

0

malformed tool calls after
fine-tuning to our agent

Contents

1	Executive Summary	3
2	Portfolio at a Glance	4
3	Methods	5
3.1	GPTQ: second-order error compensation	5
3.2	Exactness: matching the runtime's grid bit for bit	5
3.3	JANG: bits by component role	5
3.4	Riding the vendor's QAT: lossless transport into MLX	6
3.5	Where the bits go: a Δ KL-per-MB scan	6
3.6	Measuring what matters	7
4	Results	8
4.1	Gemma 4	8
4.2	Nemotron 3.5 Lightning 30B-A3B and Nemotron 3 Nano 4B	9
4.3	NemotronLabs VoiceChat 11B: real-time duplex on a laptop	11
4.4	Diffusion and audio models	11
5	Engineering We Fixed Upstream	12
6	Limitations and Lessons	12
A	All Releases	13
B	Evaluation Protocols	13
C	Reproducibility	14

1. Executive Summary

IPSupport takes an open model and fits it to a client's hardware, speed target and data. We **compress** it, so it needs a fraction of the GPU memory and answers faster, and we **fine-tune** it on the client's own tasks and formats. Nothing in the method is tied to a model size: quantization and calibration run one layer at a time and resume from checkpoints, so the limit is disk and rented GPU-hours, not GPU memory. The largest checkpoints we have shipped so far are a 31B dense and a 30B mixture-of-experts model.

The savings are concrete. Gemma 4 31B goes from 62 GB to 20.65 GB and Nemotron 30B-A3B from 65.8 GB to 16.7 GB: a model that needed a data-center GPU runs on one consumer card or a laptop. Done naively, compression quietly costs the things users notice — facts, code that compiles, a voice that answers on time. This paper shows how we avoid that, on 23 models we released openly.

Our toolkit has four parts, each validated on real checkpoints and real tasks:

- **GPTQ, exact to the runtime.** Second-order error compensation [1], with the quantization grid re-produced *bit for bit* from the inference kernel, so the correction survives packing instead of being re-rounded away.
- **JANG: bits by component role.** On mixture-of-experts models we spend bits where they are cheap and sensitive (attention, shared experts) and save them on the 93% of weights that are routed experts.
- **Riding the vendor's QAT.** When a vendor ships quantization-aware-trained weights, we transport them onto MLX *losslessly* — the same 4-bit codes as the vendor's own GGUF — instead of re-quantizing them.
- **A Δ KL-per-MB scan.** A greedy, measured choice of which few Linears earn 8 bits, under an explicit size budget.

Highlights, all published as open weights:

- **Gemma 4 31B in 20.65 GB** (from 62 GB), KL 0.021 and 94.9% top-1 agreement with the QAT master — 8 GB smaller than the community MLX build.
- **Gemma 4 12B at the same fidelity as the community QAT build with 3.1 GB less**, 1.5× faster decode, now with vision and audio after we ported the model to our MLX fork.
- **A phone-sized Gemma 4 E2B GGUF built on Google's QAT:** Google's exact 4-bit codes plus a few scanned 8-bit Linears, 2.86 GB at a PPL ratio of 0.986 to bf16 (Google's q4_0: 3.35 GB, 1.085).
- **Nemotron 3.5 Lightning 30B-A3B: PPL 5.24 in 16.7 GB** (from 65.8 GB), beating the third-party release whose bit map we reverse-engineered (5.43); bf16 is 5.11.
- **NemotronLabs VoiceChat 11B in real time on a base M5:** 185 ms → 87.7 ms per 80 ms duplex frame, 20/20 answers right.
- **Fine-tuned to an agent's tools:** a LoRA taught Nemotron the tool names and call format of our coding agent IPsupport Code — zero malformed tool calls in real sessions — and the model was then quantized on the agent's own transcripts.
- **Diffusion and music:** GPTQ cuts velocity error 17–25% at equal size on FLUX.2 klein and ACE-Step 1.5; Z-Image Turbo keeps its composition at 4 bits where plain rounding drifts.

Every claim here is backed by a script and a log in our public repository [2]; Appendix A lists all 23 releases.

2. Portfolio at a Glance

Table 1. Model families we quantized and published. Sizes are weights on disk.

Family	Formats	Headline result
Gemma 4 (E2B, E4B, 12B, 26B-A4B, 31B)	MLX, GGUF, MTP drafter	31B in 20.65 GB; 12B at community-QAT fidelity with 3.1 GB less; phone E2B GGUF in 2.86 GB on Google's QAT codes
Nemotron 3.5 Lightning 30B-A3B	MLX (+MTP head)	PPL 5.24 at 4.24 bpw vs. bf16 5.11; self-speculative MTP preserved
Nemotron 3 Nano 4B	MLX	coding-agent LoRA (attention-only), 8-bit, 4.0 GB from 7.5 GB
NemotronLabs VoiceChat 11B	MLX	real-time duplex on a base M5 (87.7 ms / 80 ms frame)
FLUX.2 klein 4B	MLX (mflux)	GPTQ 4-bit: -17% / -20% velocity error vs. RTN, 5.3 GB
Z-Image Turbo 6B	MLX (mflux)	GPTQ keeps 4-bit composition; mixed 8/4 best PSNR
ACE-Step 1.5 sft	MLX	GPTQ 4-bit: -25% velocity error vs. RTN, singable lyrics

All targets are **Apple Silicon** (MLX, via our forks of `mlx-lm` and `mlx-audio`) and **llama.cpp / ollama** (GGUF). Our MLX releases are built to run in **LLMTray**, our local AI runtime for macOS — *run local models, serve an OpenAI-compatible API, connect your coding agents and apps*.

3. Methods

3.1 GPTQ: second-order error compensation

For a linear layer with weights W and calibration inputs X , post-training quantization seeks $\widehat{W}$ on the target grid minimizing the layer's output error $\|WX - \widehat{W}X\|_2^2$. The objective's Hessian is shared by every row,

$$H = 2XX^\top.$$

GPTQ [1] quantizes one column q at a time and pushes its rounding error onto the columns not yet quantized, in proportion to the inverse Hessian:

$$\delta = \frac{w_q - \text{quant}(w_q)}{[H^{-1}]_{qq}}, \quad w_F \leftarrow w_F - \delta [H^{-1}]_{F,q},$$

with F the remaining columns. A Cholesky factorization of H^{-1} and lazy block updates make this a few GPU-minutes per layer. We calibrate on *real* data for each modality — text, COCO photographs, LibriSpeech audio, captured duplex conversations, and for diffusion models the activations of the model's own denoising loop.

Two engineering details mattered more than any hyperparameter:

- `cholesky_solve`, **not** `cholesky_inverse`. The same inverse, $\approx 30\times$ faster on a 2688-wide Hessian (0.7 s vs. 24 s). The slow call had silently inflated every GPTQ run.
- **Batched experts**. Same-shaped MoE experts are stacked into one $[E, \text{out}, \text{in}]$ problem: $5.5\times$ faster per block than a per-expert loop.

3.2 Exactness: matching the runtime's grid bit for bit

A GPTQ-corrected weight is only useful if the runtime stores exactly the code GPTQ chose. Our pipeline hands corrected weights to the stock MLX packer, which re-derives its own scales — so any mismatch between GPTQ's grid and MLX's silently re-rounds the correction away. We measured three plausible formulas before reading the kernel: they scored PPL 27.68, 6,074,737 and 17.82 on a model whose naive rounding scores 6.54. MLX's actual affine grid, per group, with $n = 2^b - 1$:

$$s = \pm \max\left(\frac{w_{\max} - w_{\min}}{n}, \varepsilon\right), \quad e = \begin{cases} w_{\min} & |w_{\min}| > |w_{\max}| \\ w_{\max} & \text{otherwise} \end{cases}, \quad q_0 = \text{round}(e/s), \quad s \leftarrow e/q_0, \quad \beta = e,$$

$$c = \text{clamp}(\text{round}((w - \beta)/s), 0, n), \quad \hat{w} = cs + \beta.$$

Implemented once and verified bit-exact against `mx.quantize` before any pod time was spent; it is what lets 3-bit GPTQ beat 3-bit rounding (PPL 6.24 vs. 6.54) at identical size and speed.

3.3 JANG: bits by component role

A third-party release of Nemotron 30B-A3B (JANG_2L-CRACK) beat our early mixes. Its `config.json` revealed why: bits assigned purely by *component type*, not by layer depth. We adopted the map and ran it through our calibrated GPTQ instead of plain rounding:

Table 2. The JANG component map (Nemotron 30B-A3B). Average 4.237 bits/weight.

Component	Bits	Why
attention q/k/v/o	8	6 of 52 layers; every token passes through
Mamba in/out projections	6	moderate share
MoE shared experts	8	one per block, always active
MoE routed up (fc1) / down (fc2)	4 / 3	128 experts × 23 blocks, ≈93% of weights
embeddings / lm_head	6 / 8	—

The lesson generalizes: *bit allocation and calibration are independent, additive levers*. The same map plus our GPTQ scored PPL 5.24 against the original's 5.43. It also exposed a bug worth stating: the MLX port names routed experts `switch_mlp.fc1/fc2`, not `up/down_proj`, so two earlier “mixed-precision” releases (ours) had never upgraded the experts at all — visible only in the packed tensor shapes on the Hub.

JANG is a MoE recipe. On dense models (Gemma E-series, VoiceChat's 9B) it degenerates to a mild 8/4 split with no structural win, and we say so in the results.

3.4 Riding the vendor's QAT: lossless transport into MLX

Google ships Gemma 4 quantization-aware-trained (QAT): a bf16 “master” and llama.cpp's q4_0 of it. We checked that the GGUF is q4_0 of the master, bit for bit, and that the community MLX ports are *not* on that grid (group 64, refit scales, MLPs at 8 bits). A q4_0 block of 32 stores a scale d and codes $c \in \{0, \dots, 15\}$ with $w = d(c - 8)$. MLX's affine 4-bit, group 32, computes $\hat{w} = s q + \beta$. Setting

$$s = d, \quad \beta = -8d$$

makes the two grids identical; we pack Google's codes directly into MLX's words. The only deliberate loss is that MLX keeps scales in bf16, so each fp16 d is rounded — under 1/64 of a step. Every 4-bit tensor's codes are checked against Google's GGUF: **275/275** (E2B), **342/342** (E4B), **328/328** (12B), **410/410** (31B) identical.

3.5 Where the bits go: a Δ KL-per-MB scan

Starting from the all-4-bit model, each candidate Linear ℓ is raised to 8 bits on its own and scored by the KL divergence it removes per megabyte it adds:

$$g_\ell = \frac{\text{KL}_{\text{base}} - \text{KL}_{\text{base}+\ell}}{\Delta \text{MB}_\ell},$$

measured on 16 held-out windows against the bf16 master. Candidates are then taken greedily in descending g_ℓ under a budget B , and the chosen build is re-measured on 128 windows from disk. On E2B the build written to disk reproduces the scan's in-memory measurement to the second decimal (PPL 64.48 vs. 64.47). What earns the bits is not the large MLPs — which the community build raises wholesale — but about a hundred small, sensitive attention and per-layer projections.

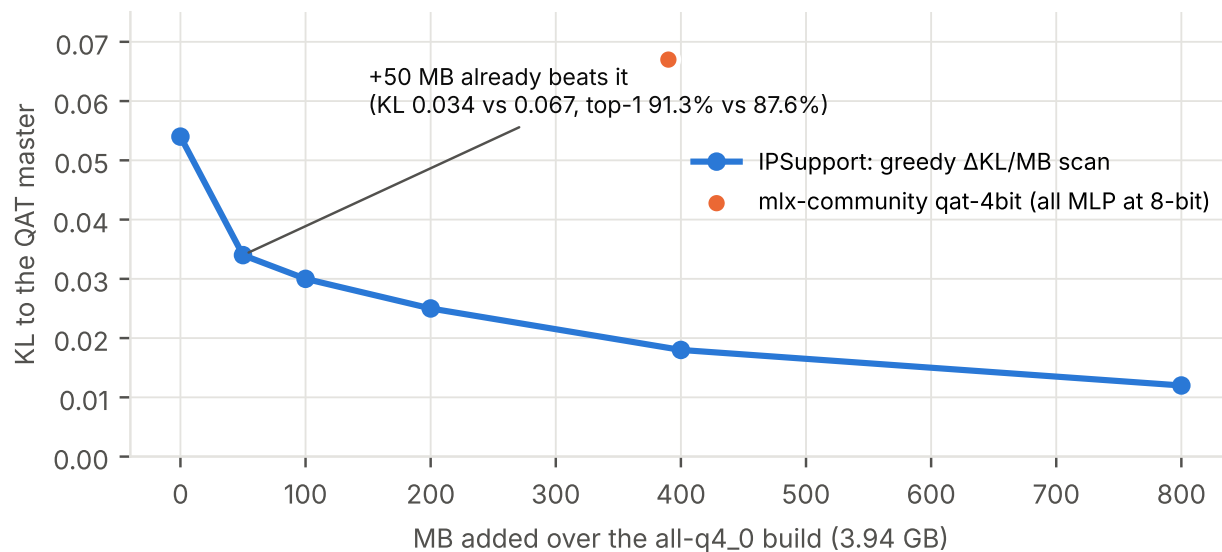


Figure 1. Gemma 4 E2B: KL to the QAT master vs. megabytes added by the greedy scan. Fifty megabytes, placed by the scan, beat a build that spends 390 MB raising every MLP (128 windows $\times$ 512 tokens).

3.6 Measuring what matters

Perplexity alone misled us repeatedly, so every release is judged by the metric its users feel:

- **Fidelity to the source:** KL divergence and top-1 agreement against the bf16 master, token by token. This is the honest measure of a quantization.
- **Capability:** code is *compiled and run* (`g++ -std=c++20`, Python), not eyeballed; lyrics are transcribed by an independent Whisper; images are compared at a fixed seed.
- **The perplexity trap.** Gemma 4 checkpoints with `attention_k_eq_v` (12B, 26B, 31B) score raw text in the thousands — 2468 in llama.cpp, 2640 in transformers, 1786 in MLX for the 31B, on Google's own weights. They fall into attractor tokens and put 6–8% of probability on control tokens. Scored as the model's *reply* with the thinking channel closed, the 26B goes from 67,363 to 26.7. We publish chat-framed numbers for these models and say why.

4. Results

4.1 Gemma 4

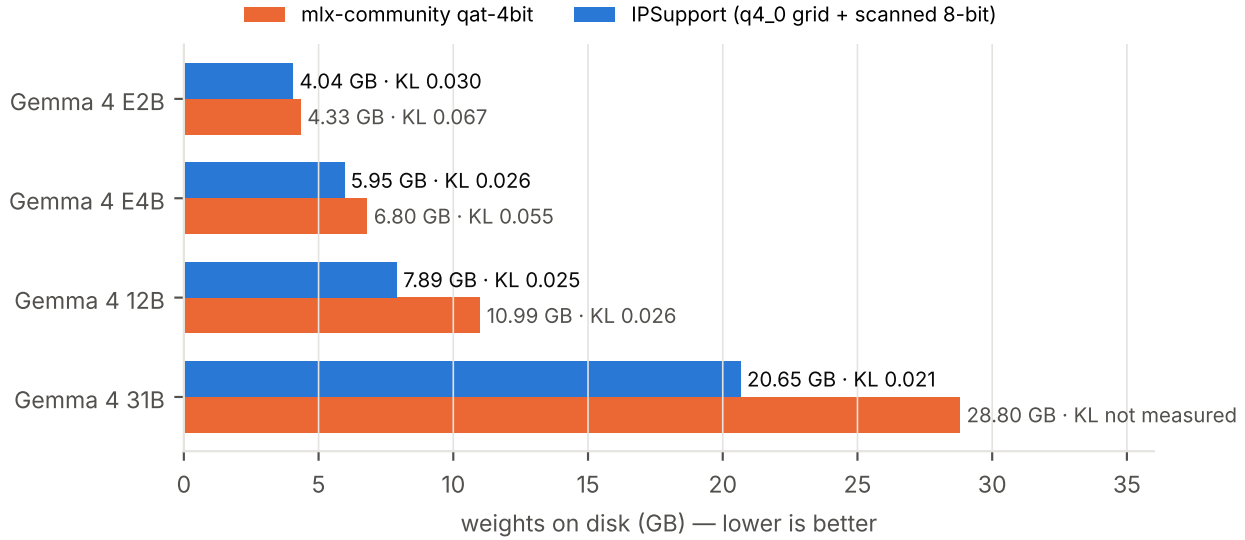


Figure 2. Gemma 4 QAT builds: weights on disk, ours vs. `mlx-community/gemma-4-*-it-qat-4bit`. KL is to the bf16 QAT master within each model (E2B/E4B raw-text windows, 12B/31B chat-framed); the community 31B was not measured.

Table 3. Gemma 4 fidelity to the QAT master (128 windows × 512 tokens). Best in bold.

Model	Build	Size	PPL	KL	Top-1
E2B	ours, q4_0 grid + 126 Linears 8-bit	4.04 GB	64.5	0.030	91.8%
	mlx-community qat-4bit	4.33 GB	66.2	0.067	87.6%
	mlx-community 4-bit RTN (no QAT)	3.6 GB	239.7	0.853	66.3%
E4B	ours, q4_0 grid	5.85 GB	44.3	0.041	90.5%
	mlx-community qat-4bit	6.80 GB	45.4	0.055	89.0%
12B [†]	ours, + 44 Linears 8-bit	7.89 GB	22.77	0.0253	93.44%
	mlx-community qat-4bit	10.99 GB	22.82	0.0259	93.35%
	mlx-community 4-bit (no QAT)	6.74 GB	26.70	0.340	78.78%
31B [†]	ours, + 35 Linears 8-bit	20.65 GB	27.82	0.0206	94.93%

[†] Chat-framed windows (Section 3.6); master PPL 22.26 (12B) and 26.94 (31B). The published E4B adds 114 Linears at 8 bits (5.95 GB, KL 0.026).

Speed on a base M5. Fewer 8-bit tensors also mean less to read per token. E2B decodes at **67.5 tok/s** vs. 54.7 for the community QAT build (+23%); E4B at **34.9** vs. 26.0 tok/s with 0.8 GB less peak memory; 12B at **14.8** vs. 9.8 tok/s (8.1 vs. 11.2 GB peak). Plain 4-bit rounding is faster still on E2B (83.7 tok/s) — at four times the perplexity.

12B, now multimodal. The 12B (`gemma4_unified`) has no vision or audio towers: raw 48-pixel patches and 640-sample audio frames go through small embedders. Our MLX fork loaded it text-only; we

ported both paths [3], fixed a cast that turned every pixel to zero once quantized, and matched mlx-
vln’s features (cosine ≥ 0.9998). The 31B passes text and vision checks (23 GB peak, measured on
the A100); it needs a 48 GB Mac (or a raised wired-memory limit).

E2B for phones. Half of E2B’s size is its per-layer embeddings. Dropping them from 6 to 4 bits costs
almost nothing (KL +0.0015, ≈ 0.6 GB saved); the scanned 8-bit Linears are kept because they halve
KL. Google also ships a “mobile” QAT (int2/4/8 weights); we converted it exactly to MLX (2.57 GB,
122.5 tok/s) but it needs its int8 *activations* — without them PPL rises from 41 to 65 and the audio
tower goes deaf — so it suits an NPU runtime, not MLX or llama.cpp.

Table 4. Gemma 4 E2B phone builds. GGUF: llama.cpp, 32 chunks of 512, against our bf16 GGUF of the master
(PPL 42.84).

Build	Size	PPL	vs. bf16	Decode (M5)
ours, GGUF (Q4_0 + PLE Q4_K + Q8_0 Linears)	2.86 GB	42.24	×0.986	—
Google gemma-4-E2B_q4_0-it.gguf	3.35 GB	46.47	×1.085	—
ours, MLX (PLE 4-bit, towers 4-bit)	3.10 GB	36.44 [‡]	—	70.2 tok/s

[‡]128 chat windows, master 36.04; KL 0.0242, top-1 92.48%. All 149 of our Q4_0 tensors are byte-identical to Google’s; the
gain is the Q8_0 Linears.

26B-A4B (MoE): GPTQ JANG, GGUF and a speculative drafter. The 128-expert 26B is quantized with
per-expert GPTQ (calibration rows bucketed by the real routing, all 128 experts solved in one batch),
attention at 8 bits, experts at 4, routers untouched: **15 GB from 51.6 GB**. The GGUF build (imatrix on
a broad corpus, attention Q5_K, experts IQ3_S) is 12.5 GB. Google’s 4-layer MTP drafter had no MLX
support; we implemented it [3] and published it at 8 bits (446 MB): **35.8 → 52.9 tok/s** on a short prompt,
32.7 → 41.7 at 3.9k context, with the main model’s output unchanged.

4.2 Nemotron 3.5 Lightning 30B-A3B and Nemotron 3 Nano 4B

Table 5. Nemotron 30B-A3B (hybrid Mamba-2 / attention / 128-expert MoE, 52 layers). The PPL column relative
to bf16 is the one to read: uniform 3-bit is an extreme quant.

Build	PPL	vs. bf16	Size	bits/weight
bf16 (reference)	5.11	—	65.8 GB	16
JANG map + our GPTQ (recommended)	5.24	+2.5%	16.7 GB	4.237
JANG_2L-CRACK (third party, presumed RTN)	5.43	+6.3%	16 GB	≈ 3.73 (their label)
uniform 3-bit, our GPTQ (extreme)	6.24	+22%	13.8 GB	3.5
uniform 3-bit, naive rounding (extreme)	6.54	+28%	13 GB	3.5

Speed is set by the bit map’s structure, not its average. Uniform 3-bit decodes at 65–69 tok/s; both
JANG builds, ours and the third party’s, at 42–43 tok/s despite different calibration and average bits.
JANG spends its bits on the dense components that every token touches, so “cheap on disk” and
“cheap per token” are opposite properties on this architecture. But speed is the only thing uniform
3-bit wins: it is an **extreme quant**, +22% perplexity over bf16 against JANG’s +2.5%, and in real use its
answers degrade markedly. We keep it on the Hub as the latency end of the frontier and recommend
the JANG build for anything that has to be right.

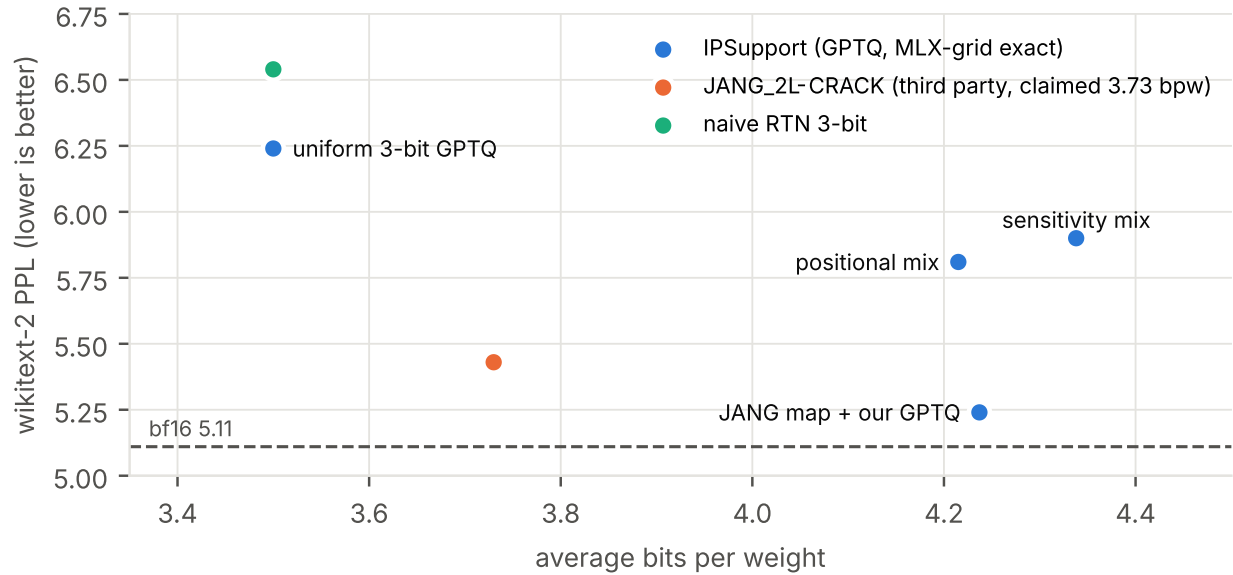


Figure 3. Nemotron 3.5 Lightning 30B-A3B, wikitext-2 perplexity vs. bits per weight (20 × 512 tokens). The JANG component map run through our grid-exact GPTQ lands closest to bf16.

Why a LoRA: our agent's tools. Both Nemotrons carry a LoRA for **IPSupport Code**, our local coding agent, and it exists to fix one concrete failure: driving the agent, the base model kept getting our tools wrong — wrong tool names, wrong parameter names, malformed JSON in the call. We trained on the real sessions where it failed, plus synthetic conversations covering the same patterns, merged the adapter into bf16, and only then quantized, calibrating GPTQ on the agent's own tool-call transcripts. Result in real use: **zero malformed tool calls across a session**, and on held-out tool-call data PPL 2.464 → 2.379 (−3.5%) for +0.8% on generic prose — the intended trade for a model that lives inside an agent.

...and what it taught us. Two findings came only from running the code the model wrote: training attention and MLP adapters *together* on the 4B dropped C++ compile-and-run from 6/9 to 0/9 at any rank or length, while either half alone matched the base — so the shipped 4B adapter is attention-only. And GPTQ, calibrated on text with no C++ in it, scored 2/9 where plain 8-bit rounding scored 4/9 (bf16: 6/9): calibration preserves what the calibration set contains, so the 4B ships with the method that won. The fine-tune also leaves a reflex: given tools in a plain chat, the 30B reaches for one on a greeting 10 times in 20 (the base: 0). Right for an agent, wrong for small talk; LLMTray sends a system rule that cuts it on questions from 14/20 to 4–6/20, and the cards say so.

Keeping the MTP head. The 30B ships a 1.34B-parameter multi-token-prediction head that every public quant had silently dropped: transformers discards `mtp.*` on load, before any merge or quantization runs. We extract it from the untouched checkpoint and splice it back into the finished MLX model; self-speculative decoding is bit-exact against greedy. Along the way we fixed two decoding bugs in the MTP path [3]: a fused single-step SSM kernel that corrupted the Mamba state on every rejected draft, and an unchunked prefill that made 400-token prompts run out of memory (now flat at 18.4 GB up to 4000 tokens).

4.3 NemotronLabs VoiceChat 11B: real-time duplex on a laptop

VoiceChat is a full duplex voice model: a streaming FastConformer encoder, the Nemotron Nano 9B hybrid LLM, a T5Gemma TTS and a 22 kHz codec. It must produce one 80 ms audio frame in under 80 ms on a base M5 — fanless, 19 GB GPU budget.

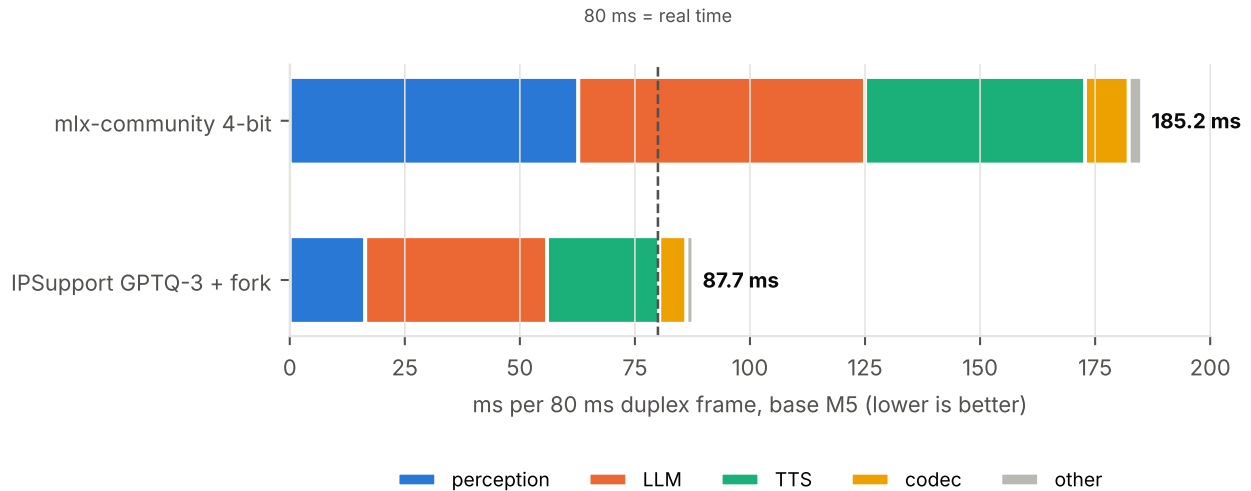


Figure 4. Milliseconds per 80 ms frame, 20 spoken questions, idle base M5. Baseline: the community 4-bit checkpoint. Ours: GPTQ 3-bit LLM plus runtime fixes in our `mlx-audio` fork.

GPTQ 3-bit on the LLM, calibrated on 256 captured real conversations, answered **20/20** questions with a reply word-error rate of 0.043, the lowest among the variants that answered every question. The rest came from the runtime: perception was running in float32 because mel frames arrived that way (51 → 17 ms after the cast; 13.6 ms compiled), and the TTS head computed every mixture component instead of the sampled one. The LLM now sits at the memory-bandwidth floor (≈105 GB/s effective). A second release adds 8-bit speech towers (6.43 GB, 83.3 ms cold), and pausing the TTS while the model listens brings an average frame to 69.1 ms (real-time factor 0.86). We tried a 2-bit MLP: 4 ms faster, wrong answers (“blue + yellow = gray”). It was not shipped.

4.4 Diffusion and audio models

For diffusion models the calibration signal is the model’s own sampling: we hook every target Linear through real denoising steps and sum each Hessian over every token of every step.

Table 6. GPTQ vs. plain rounding (RTN) at equal size. Velocity error is teacher-forced $\|v_q - v_{bf16}\| / \|v_{bf16}\|$, so trajectory drift does not compound into it.

Model	Build	GPTQ vs. RTN
FLUX.2 klein 4B	4-bit, 5.3 GB	velocity error 0.202 vs. 0.244 (–17%) text-to-image, 0.136 vs. 0.171 (–20%) edit; lower on all 9 held-out samples; PSNR 19.6 vs. 18.7 dB
ACE-Step 1.5 sft (music)	4-bit DiT, 1.65 GB	velocity error 10.4% vs. 13.9% (–25%), condition encoder 2.7% vs. 5.0% (–47%)
Z-Image Turbo 6B	4-bit / mixed 8+4	PSNR 17.1 vs. 16.6 dB; at 4 bits RTN shifts the whole composition, GPTQ holds it; attention-8 / FFN-4 reaches 17.4 dB for +0.8 GB

FLUX.2 klein. The text encoder is stock Qwen3-4B, byte for byte, and the pipeline only reads hidden states 9, 18 and 27 — so layers 27–35 never touch the image. We drop them: prompt embeddings stay bit-identical and 1GB disappears. With the encoder at 8 bits and the transformer at GPTQ 4 bits, the model generates and edits in 5.3 GB (bf16: 16 GB), 7.3 GB peak at 1024².

ACE-Step. Run through the stock MLX audio stack, the sft model sang gibberish. Two causes, both measured with an independent Whisper: the planner’s audio hints break the sft DiT, and the classifier-free-guidance branch encoded zeros instead of the trained null embedding. Fixed, it sings intelligibly: word-error rate 0.22 in bf16, 0.24 at 8 bits, 0.29 with GPTQ 4-bit, peak memory 10.6 / 8.6 / 7.5 GB.

5. Engineering We Fixed Upstream

Quantization work runs into the runtime. These fixes are merged in our forks or published as patches:

- **Gemma 4 KV-shared layers crashed with a quantized KV cache** (a shared layer handed a quantized tuple to the unquantized attention path) — a production crash in LLMTray.
- **Gemma 4 MTP drafter for MLX**, with an exact cache rollback past the 1024-token sliding window, and a RoPE offset bug that made the drafter useless whenever the server reused a cached prefix (+48% offline, nothing in the app until fixed).
- **Nemotron-H MTP**: the speculative step left the cache one token short, dropping the last token of every assistant reply from multi-turn context; plus the SSM rollback and prefill-memory fixes above.
- **Gemma 4 GGUF that chats correctly**: a tokenizer conversion crash, control tokens tagged as text (they leaked <|channel>thought into replies), and an ollama RENDERER/PARSER Modelfile that a Hub repo cannot express on its own.
- **Three llama.cpp bugs mistagging dense Nemotron-H as MoE** after a `save_pretrained` round trip; conversion “succeeded” and the file would not load.
- `gemma4_unified` **vision and audio** in MLX (Section 4.1).

6. Limitations and Lessons

We publish our negative results too; several shaped the methods above.

- **QAT beats post-training quantization.** Where a vendor ships QAT weights we transport them rather than re-quantize; our own PTQ of the non-QAT E2B was not released.
- **Calibration preserves what it sees.** GPTQ helps on the calibration distribution and can cost capabilities outside it (Section 4.2). We calibrate per modality and verify with task tests.
- **Perplexity is not a quality measure on its own.** It missed a broken <think> close, a C++ regression, and it reports thousands for healthy Gemma 4 models.
- **2 bits cost facts.** On VoiceChat’s LLM every 2-bit MLP variant answered questions wrong that 3 bits answered right.
- **Laptops throttle.** On a fanless M5 the same build measured 84 ms and then 117 ms in back-to-back runs. Our timings come from cool, idle machines in alternated order, and real long sessions will run slower than the cold figures.
- **Raw sizes are not everything.** The 31B at 20.65 GB still exceeds a 32 GB Mac’s GPU budget once the KV cache grows; we state memory requirements on every card.

A. All Releases

Table 7. Public releases on Hugging Face ([roman220220/...](https://huggingface.co/roman220220/)). Sizes are weights; downloads are the Hub's 30-day count at the time of writing (3,000+ in total).

Repository	Format	Contents
gemma-4-31B-it-qat-mlx	MLX	20.65 GB, text + vision
gemma-4-12B-it-qat-mlx	MLX	7.89 GB, text + vision + audio
gemma-4-E4B-it-qat-mlx	MLX	5.95 GB, text + vision + audio
gemma-4-E2B-it-qat-mlx	MLX	4.04 GB, text + vision + audio
gemma-4-E2B-it-qat-phone-mlx	MLX	3.10 GB, phone build
gemma-4-E2B-it-qat-phone-GGUF	GGUF	2.86 GB + Google's mmproj
gemma-4-26B-A4B-it-gptq-mlx-jang	MLX	≈15 GB, MoE, text + vision
gemma-4-26B-A4B-it-GGUF-jang-imatrix	GGUF	12.5 GB + mmproj + MTP drafter
gemma-4-26B-A4B-it-assistant-mlx-8bit	MLX	446 MB MTP drafter
Nemotron-3.5-Lightning-30B-A3B-JANG-GPTQ- ipsupport-code-lora-MTP	MLX	coding-agent LoRA + MTP head
Nemotron-3.5-Lightning-30B-A3B-JANG-GPTQ- ipsupport-code-lora	MLX	coding-agent LoRA
nemotron-30b-a3b-gptq-jang-component	MLX	16.7 GB, JANG + GPTQ, 4.237 bpw
nemotron-30b-a3b-gptq3bit-g64	MLX	uniform 3-bit GPTQ, 13.8 GB
NVIDIA-Nemotron-3-Nano-4B-JANG-GPTQ-ipsupport- code-lora	MLX	coding-agent LoRA, 8-bit
NemotronLabs-VoiceChat-11B-gptq-mlx-3bit	MLX	real-time duplex voice
NemotronLabs-VoiceChat-11B-gptq-mlx-mixed	MLX	+ 8-bit speech towers, 6.43 GB
flux2-klein-4b-mlx-mixed	MLX	generate + edit, 5.3 GB
z-image-turbo-gptq-mlx-4bit	MLX	5.5 GB
z-image-turbo-gptq-mlx-8bit	MLX	≈10 GB
z-image-turbo-gptq-mlx-mixed	MLX	attention 8 / FFN 4, 6.3 GB
ACE-Step1.5-sft-MLX-bf16	MLX	reference
ACE-Step1.5-sft-MLX-8bit	MLX	8-bit DiT
ACE-Step1.5-sft-MLX-gptq-4bit	MLX	GPTQ 4-bit DiT

B. Evaluation Protocols

- **Language models:** wikitext-2 test, BOS at the start of every window (Gemma needs it); KL and top-1 agreement against the bf16 source checkpoint over 128 windows of 512 tokens. Chat-only checkpoints are scored as the assistant's reply with the thinking channel closed.
- **Code:** 3 seeds × 3 temperatures per prompt, the extracted code compiled with `g++ -std=c++20` or executed in Python; pass/fail counted.
- **Speech:** 20 spoken questions; keyword accuracy and reply word-error rate from an independent Whisper; per-frame timings on an idle base M5 after a 5-minute cool-down, order alternated.
- **Music:** 3 songs × 4 seeds, 30 s each, Whisper word-error rate against the requested lyrics.
- **Images:** fixed seeds, PSNR against bf16 and teacher-forced velocity error on held-out prompts and edits.

C. Reproducibility

Every release is produced by a resumable pipeline in our repository [2] — `qat_mlx_pipeline.sh` (Gemma QAT), `gemma4_mlx_pipeline.sh` and `gemma4_26b_gguf_pipeline.sh` (GPTQ JANG and GGUF), `run_pipeline.sh` (Nemotron) — with calibration on a rented A100 and conversion checked on a Mac. Each project's `docs/FINDINGS.md` records every measurement, including the ones that went wrong.

References

- [1] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. ICLR 2023.
- [2] IPSupport LLC. *quant-ternary: quantization pipelines and findings*.
<https://github.com/rromenskyi/quant-ternary>
- [3] IPSupport LLC. *mlx-lm fork with Gemma 4 and Nemotron-H support*.
<https://github.com/ip-support-llc/mlx-lm>

IPSupport **LLC**

Senior engineering for cloud, voice and AI systems.

7533 S Center View Ct #6055
West Jordan, UT 84084, USA

+1 (385) 297-0034
hello@ipsupport.us

ipsupport.us

LLMTray — Local AI Runtime for macOS: run local models, serve
an OpenAI-compatible API, connect your coding agents and apps.

IPSupport Code — your AI coding agent for real repositories.